

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO 5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems.(L4,L5)

NAME: K.DEEPIKA

REG.NO:192371042

SUBJECT:INTERNET PROGRAMMING

1. SOAP-Based Hospital Management Web Service

Introduction

A hospital management system can use a **SOAP (Simple Object Access Protocol) based web service** to allow different departments such as reception, outpatient department, laboratory, pharmacy, and billing to exchange patient information in a standardized and secure manner.

SOAP uses **XML-based messages** for communication between a client and a server. It is suitable for hospital applications because healthcare systems require **structured communication, reliability, security, and interoperability**.

Basic Architecture

Hospital Client / Department

|
| SOAP Request (XML)



SOAP Web Service



| Process Request



Hospital Database



| SOAP Response (XML)



Hospital Client / Department

Main Operations

The service can provide operations such as:

Operation	Purpose
addPatient()	Registers a new patient
getPatient()	Retrieves patient details
updatePatient()	Updates patient information
scheduleAppointment()	Books an appointment
getAppointment()	Retrieves appointment details
getMedicalRecords()	Retrieves patient's medical records
updateMedicalRecord()	Updates medical information

SOAP Message Structure

A SOAP message consists mainly of:

1. **Envelope**
2. **Header**
3. **Body**

<soap:Envelope

xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">

<soap:Header>

<Authentication>

<Username>hospitalUser</Username>

<Token>ABC123</Token>

</Authentication>

</soap:Header>

<soap:Body>

<AddPatient>

<Patient>

<PatientID>P1001</PatientID>

<Name>Arun Kumar</Name>

<Age>35</Age>

<Gender>Male</Gender>

<Phone>9876543210</Phone>

</Patient>

</AddPatient>

</soap:Body>

</soap:Envelope>

1. Envelope

The **Envelope** is the root element of every SOAP message.

It identifies the message as a SOAP message and contains the Header and Body.

<soap:Envelope>

...

</soap:Envelope>

2. Header

The **Header** contains additional information required for processing the request.

For example:

- Authentication
- Authorization
- Security tokens
- Transaction information
- Routing information

<soap:Header>

<Authentication>

<Username>hospitalUser</Username>

<Token>ABC123</Token>

</Authentication>

</soap:Header>

For a real hospital system, sensitive information should be protected using appropriate security mechanisms such as **HTTPS/TLS and WS-Security**, rather than sending credentials as plain XML.

3. Body

The **Body** contains the actual request or response.

For example:

<soap:Body>

 <GetMedicalRecords>

 <PatientID>P1001</PatientID>

 </GetMedicalRecords>

</soap:Body>

The server processes the request and returns:

<soap:Envelope>

 <soap:Body>

 <GetMedicalRecordsResponse>

 <PatientID>P1001</PatientID>

 <Diagnosis>Fever</Diagnosis>

 <Treatment>Medication</Treatment>

 <Doctor>Dr. Kumar</Doctor>

 </GetMedicalRecordsResponse>

 </soap:Body>

</soap:Envelope>

Data Exchange Process

The communication takes place as follows:

1. The hospital department sends a request to the SOAP endpoint.
2. The client creates a SOAP XML message.
3. Authentication/security information is included in the SOAP Header when required.
4. The requested operation and data are placed in the SOAP Body.
5. The SOAP message is transmitted to the server, normally over HTTPS.
6. The server validates the request.
7. The server processes the requested operation.
8. The hospital database is accessed.
9. The server creates a SOAP response.
10. The client receives and processes the response.
11. If an error occurs, the server returns a SOAP Fault.

SOAP Fault

Errors can be represented using a SOAP Fault message.

```
<soap:Fault>
```

```
  <faultcode>soap:Client</faultcode>
```

```
  <faultstring>Patient ID is required</faultstring>
```

```
</soap:Fault>
```

Why SOAP is Suitable for a Hospital System

SOAP is suitable because:

- **Interoperability:** Different hospital departments and systems can communicate even when developed using different programming languages.
- **Structured messages:** XML provides a standardized structure.
- **Security:** SOAP can use WS-Security for message-level security, while HTTPS/TLS protects communication in transit.
- **Reliability:** SOAP supports standards and extensions for reliable messaging and transactions.
- **Error handling:** SOAP Fault provides a standardized mechanism for reporting errors.
- **WSDL support:** WSDL clearly describes available operations and message formats.
- **Enterprise support:** SOAP is widely suited to complex enterprise environments where strict contracts and standards are important.

Conclusion

Therefore, a SOAP-based hospital management service provides a **standardized, secure, reliable, and interoperable mechanism** for exchanging patient, appointment, and medical-record information between hospital departments.

2. WSDL Structure for an Enterprise Web Service

Introduction

WSDL (Web Services Description Language) is an XML-based language used to describe a web service.

It acts as a **contract between the service provider and service consumer**. It tells external systems:

- What operations are available
- What data must be sent

- What data will be returned
- How messages are structured
- Which communication protocol is used
- Where the service is located

Main Components of WSDL

A WSDL document generally contains:

WSDL

|

+-- Types

|

+-- Messages

|

+-- PortType

|

+-- Binding

|

+-- Service

Example WSDL Structure

<definitions

name="EmployeeService"

targetNamespace="http://example.com/employee"

xmlns="http://schemas.xmlsoap.org/wsdl/"

xmlns:tns="http://example.com/employee"

xmlns:xsd="http://www.w3.org/2001/XMLSchema"

xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/">

<!-- 1. Types -->

<types>

<xsd:schema

targetNamespace="http://example.com/employee">

<xsd:element name="getEmployee">

<xsd:complexType>

<xsd:sequence>

<xsd:element name="employeeID"

type="xsd:int"/>

</xsd:sequence>

</xsd:complexType>

</xsd:element>

</xsd:schema>

</types>

<!-- 2. Messages -->

<message name="GetEmployeeRequest">

<part name="parameters"

```
        element="tns:getEmployee"/>
</message>

<message name="GetEmployeeResponse">
    <part name="result"
        type="xsd:string"/>
</message>

<!-- 3. PortType -->

<portType name="EmployeePortType">

    <operation name="getEmployee">
        <input message="tns:GetEmployeeRequest"/>
        <output message="tns:GetEmployeeResponse"/>
    </operation>

</portType>

<!-- 4. Binding -->

<binding name="EmployeeSoapBinding"
    type="tns:EmployeePortType">

    <soap:binding
```

```
transport="http://schemas.xmlsoap.org/soap/http"
style="document"/>
```

```
<operation name="getEmployee">
```

```
  <soap:operation
    soapAction="getEmployee"/>
```

```
  <input>
```

```
    <soap:body use="literal"/>
```

```
  </input>
```

```
  <output>
```

```
    <soap:body use="literal"/>
```

```
  </output>
```

```
</operation>
```

```
</binding>
```

```
<!-- 5. Service -->
```

```
<service name="EmployeeService">
```

```
  <port name="EmployeeSoapPort"
```

```
    binding="tns:EmployeeSoapBinding">
```

```
<soap:address
    location="https://example.com/EmployeeService"/>
</port>

</service>

</definitions>
```

Explanation of Components

1. Types

The **Types** section defines the data types used by the service.

It generally uses **XML Schema (XSD)**.

Example:

```
<xsd:element name="employeeID"
    type="xsd:int"/>
```

It specifies that employeeID must contain an integer.

Therefore:

Types → Defines the structure and data types of information.

2. Message

The **Message** element describes the data exchanged between client and server.

Example:

```
<message name="GetEmployeeRequest">
```

```
<part name="parameters"
      element="tns:getEmployee"/>
</message>
```

A response message can also be defined:

```
<message name="GetEmployeeResponse">
  <part name="result"
        type="xsd:string"/>
</message>
```

Therefore:

Message → Defines the input and output data.

3. PortType

The **PortType** defines the abstract interface of the web service.

It specifies the available operations.

```
<portType name="EmployeePortType">
  <operation name="getEmployee">
    <input message="tns:GetEmployeeRequest"/>
    <output message="tns:GetEmployeeResponse"/>
  </operation>
</portType>
```

For example, a service could have:

EmployeePortType

|

```
+-- getEmployee()  
  
+-- updateEmployee()  
  
+-- deleteEmployee()
```

Therefore:

PortType → Defines what operations the service provides.

4. Binding

The **Binding** section specifies how the abstract operations are implemented for communication.

For a SOAP service:

```
<soap:binding  
  
  transport="http://schemas.xmlsoap.org/soap/http"  
  
  style="document"/>
```

It defines details such as:

- SOAP
- HTTP transport
- Document/RPC style
- Encoding
- SOAP actions

Therefore:

Binding → Defines how the service communicates.

5. Service

The **Service** element provides the actual endpoint where the service can be accessed.

```
<service name="EmployeeService">  
  <port name="EmployeeSoapPort"  
    binding="tns:EmployeeSoapBinding">  
    <soap:address  
      location="https://example.com/EmployeeService"/>  
    </port>  
</service>
```

Therefore:

Service → Defines where the service is available.

Relationship Between Components

The complete relationship can be represented as:

Types



Defines data structures



Messages



Defines request/response



PortType



Defines operations



Binding



Defines communication protocol



Service



Defines actual endpoint

Importance of WSDL

WSDL is important for service integration because:

1. **Provides a clear service contract**
2. **Allows different systems to understand the service**
3. **Defines operations and data formats**
4. **Specifies communication protocols**
5. **Specifies service endpoints**
6. **Supports interoperability**
7. **Allows automatic generation of client code in many SOAP toolchains**
8. **Reduces integration errors**

Conclusion

WSDL acts as a **formal interface description** between a web service provider and consumer. It allows external applications to understand exactly **what the service does, what data it accepts, how it communicates, and where it can be accessed**.

3. Debugging Problems in SOAP Communication

Introduction

In a SOAP-based system, communication takes place through XML messages exchanged between a client and a server.

Client

|

| SOAP Request

↓

Endpoint / Web Server

|

↓

SOAP Service

|

↓

Database / Application

|

| SOAP Response

↓

Client

If requests are not processed correctly or responses are delayed/incorrect, different parts of this communication path must be checked systematically.

Possible Problems

1. SOAP Message Formatting Errors

SOAP requires a correctly structured XML document.

Incorrect:

```
<soap:Envelope>

  <soap:Body>

    <GetPatient>

      <PatientID>P1001

    </GetPatient>

  </soap:Body>

</soap:Envelope>
```

The PatientID element is not properly closed.

Correct:

```
<soap:Envelope>

  <soap:Body>

    <GetPatient>

      <PatientID>P1001</PatientID>

    </GetPatient>

  </soap:Body>

</soap:Envelope>
```

Therefore, the first step is to validate the XML syntax.

2. Incorrect Namespace

SOAP and application elements must use the namespaces expected by the service.

Example:

<soap:Envelope

xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">

If the client uses an incorrect namespace, the server may fail to recognize the operation.

For example:

Expected:

<http://example.com/patient>

Received:

<http://example.org/patient>

Even though the element names look identical, they represent different XML namespaces.

3. Incorrect Endpoint

The client must send the request to the correct service endpoint.

Example:

Correct:

<https://hospital.com/services/PatientService>

Possible problems include:

- Wrong URL
- Wrong port
- Service not running
- DNS problem
- Firewall blocking the connection
- Incorrect HTTP configuration

4. Incorrect SOAPAction

Some SOAP services use the SOAPAction HTTP header to identify the requested operation.

For example:

SOAPAction: "getPatient"

If the client sends an incorrect action, the server may reject or incorrectly route the request.

5. WSDL Mismatch

The client may be using an outdated or incorrect WSDL.

For example:

Client expects:

GetPatient(PatientID)

Server expects:

GetPatient(PatientID, HospitalID)

This can result in request-processing errors.

The client and server should use compatible service definitions.

6. Authentication and Authorization Errors

If the service requires authentication, an invalid or expired security token can cause failure.

Possible problems:

- Invalid username/password
- Expired token
- Missing security header
- Insufficient permissions
- Incorrect WS-Security configuration

7. Server-Side Errors

The SOAP request may be correct but the server may encounter an internal problem.

Examples:

- Database unavailable
- Invalid SQL query
- Application exception
- Missing record
- Server overload

These should be returned using an appropriate SOAP Fault rather than an incorrect normal response.

8. Network and Timeout Problems

Delayed responses can be caused by:

- Network latency
- Server overload
- Database delays
- Incorrect timeout configuration
- Firewall/proxy problems
- Service deadlock or processing failure

The request and response timestamps should be recorded to identify where the delay occurs.

Debugging Approach

A systematic debugging process can be followed.

Step 1: Check the Endpoint

Verify:

Client → Correct URL → Correct Server → Correct SOAP Service

Use tools such as a browser/API testing tool or network monitoring tools to verify connectivity.

Step 2: Validate the XML

Check:

- Opening and closing tags

- XML syntax
- Required elements
- Correct data types
- Proper encoding

Example:

```
<PatientID>P1001</PatientID>
```

should match the expected schema.

Step 3: Verify Namespaces

Compare the namespaces in the request with those defined by the WSDL.

```
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
```

```
xmlns:p="http://example.com/patient"
```

Incorrect namespaces should be corrected.

Step 4: Compare Request with WSDL

Check:

Operation name



Input message



Element names



Data types



Required fields

The SOAP request must follow the service contract defined by WSDL.

Step 5: Inspect SOAP Headers

Verify:

- Authentication information
- Security tokens
- SOAPAction
- Transaction information
- Required WS-Security elements

Step 6: Inspect Server Logs

Server logs should be checked for:

Request received



Authentication successful



Operation identified



Database request



Database response



SOAP response generated

This helps determine exactly where processing fails.

Step 7: Check Database/Application Logic

If the SOAP request reaches the server correctly but returns incorrect information, check:

- Database connection
- SQL queries
- Stored procedures
- Data validation
- Business logic
- Exception handling

Step 8: Test SOAP Fault Handling

A proper SOAP service should return meaningful errors.

Example:

```
<soap:Fault>
```

```
  <faultcode>soap:Server</faultcode>
```

```
  <faultstring>
```

```
    Unable to retrieve patient records
```

```
  </faultstring>
```

```
</soap:Fault>
```

The client should detect the Fault and handle it appropriately instead of treating it as a successful response.

Step 9: Check Response Data

Verify that the response:

- Contains the expected operation response
- Uses correct namespaces
- Contains correct data types
- Contains all required fields
- Is valid XML

Step 10: Test Again

After correcting the problem:

Create Request



Validate XML



Send Request



Server Processes Request



Validate Response



Check Expected Result

Repeat the test with valid and invalid inputs.

Error Handling Strategy

A reliable SOAP system should distinguish different types of errors:

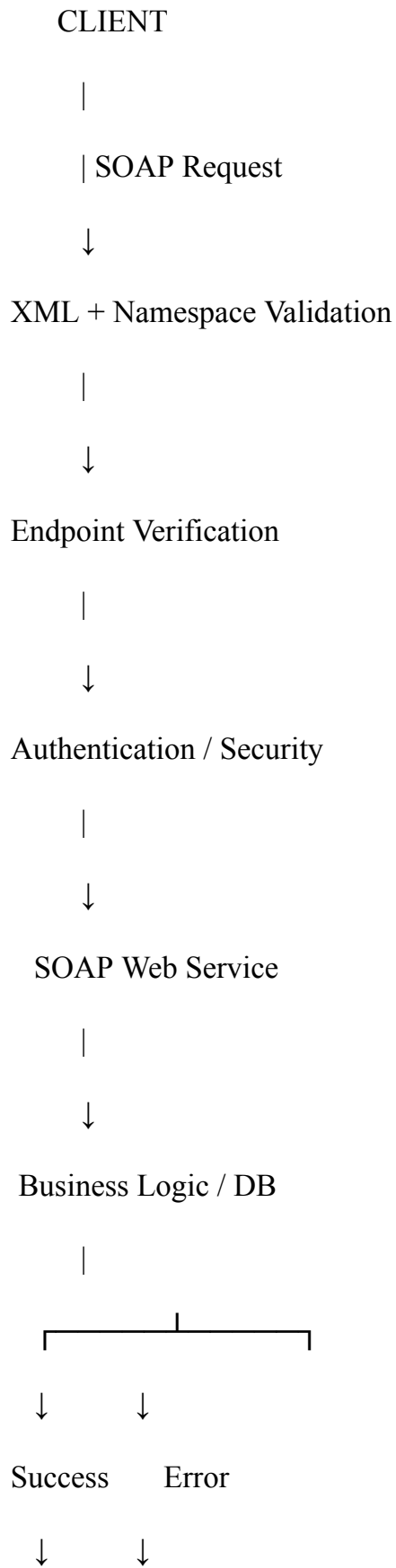
Error	Example	Handling
Client Error	Invalid patient ID	SOAP Fault
Authentication Error	Invalid token	Security fault
Server Error	Database failure	Server SOAP Fault
Network Error	Timeout	Retry/timeout handling
Validation Error	Missing field	Validation fault
Service Error	Wrong operation	SOAP Fault

How the Corrected System Ensures Reliability

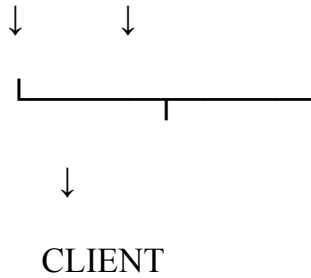
After debugging:

1. XML messages are validated.
2. Namespaces match the WSDL.
3. Endpoint URLs are verified.
4. SOAP operations match the service contract.
5. Authentication and authorization are checked.
6. Server-side logs monitor requests and responses.
7. SOAP Faults provide structured error information.
8. Appropriate timeout and retry mechanisms can handle temporary failures.
9. Database and application errors are handled properly.
10. Response data is validated before being accepted.

Final Flow



SOAP Response SOAP Fault



Conclusion

SOAP communication problems can usually be resolved by checking the communication path systematically—from **endpoint and XML formatting to namespaces, WSDL, authentication, server processing, database operations, and response validation**. Proper SOAP Fault handling, logging, validation, timeout management, and security mechanisms ensure that the corrected system provides **reliable and predictable communication**.